

Lecture 3.2: NumPy

XCS229 Machine Learning

[Lecturer (i.e. Tengu Ma or Chris Re)]

I. NumPy

NumPy is primarily used for linear algebra operations and array manipulation, significantly speeding up computations (a nice vector and matrix manipulation package).

To use NumPy, one must first import the library using `import numpy as np`.

```
[1]: import numpy as np
```

Array initialization

Initialize from existing list. If you type is not consistent, numpy will give you weird result.

When creating an array, NumPy assigns a data type (dtype) to the array based on the contents of the list. If all elements are integers, the dtype will be `Int64`. However, if a float is included, all elements will be converted to floats.

```
[2]: from_list=np.array([1, 2, 3])
      pp.pprint(from_list)
      print(f'\t Data type of integer is {from_list.dtype}')
```

```
array([1, 2, 3])
      Data type of integer is int64
```

```
[3]: from_list_2d=np.array([[1, 2, 3.0], [4, 5, 6]])
      pp.pprint(from_list_2d)
      print(f'\t Data type of float is {from_list_2d.dtype}')
```

```
array([[1., 2., 3.],
      [4., 5., 6.]])
      Data type of float is float64
```

Mixed types can lead to unexpected dtypes like U21, indicating a need for consistent data types.

```
[4]: from_list_bad_type=np.array([1, 2, 3, 'a'])  
      pp.pprint(from_list_bad_type)
```

```
array(['1', '2', '3', 'a'], dtype='<U21')
```

Initialize with ones, zeros ,or identity matrix:

- Initializing an array with ones:

```
[5]: print(np.ones(3))
```

```
[1.  1.  1.]
```

- Initializing a 2D array with ones:

```
[6]: print(np.ones((3,3)))
```

```
[[1.  1.  1.]  
 [1.  1.  1.]  
 [1.  1.  1.]]
```

- Initializing an array with zeros:

```
[7]: print(np.zeros(3))
```

```
[0.  0.  0.]
```

- Initializing a 2D array with zeros:

```
[8]: print(np.zeros((3,3)))
```

```
[[0.  0.  0.]  
 [0.  0.  0.]  
 [0.  0.  0.]]
```

- Initializing an identity matrix:

```
[9]: print(np.eye(3))
```

```
[[1. 0. 0.]
 [0. 1. 0.]
 [0. 0. 1.]]
```

Sampling over uniform distribution on $[0, 1)$:

The function `np.random.random()` is used to generate random numbers sampled from a uniform distribution over the interval $[0, 1)$. This means the numbers will be evenly distributed between 0 and 1, excluding 1.

```
[10]: print(np.random.random(3))
```

```
[0.77902369 0.81865665 0.3503008]
```

You can also specify the shape of the output array as a tuple to generate multiple random samples, for example:

```
[11]: print(np.random.random((2,2)))
```

```
[[0.05704649 0.65507107]
 [0.50732387 0.66362862]]
```

Here, we get a 2×2 array of random numbers sampled from the uniform distribution.

Sampling over normal distribution:

For sampling from a standard normal distribution (mean = 0, standard deviation = 1), you can use `np.random.randn()`. This function generates random samples from a normal distribution with the specified dimensions.

The example below generates a 3×3 array of random numbers from a standard normal distribution, with each element drawn independently.

```
[12]: print(np.random.randn(3, 3))
```

```
[[ 0.20327001  1.86688672 -0.61089097]
 [ 0.27371701 -0.21164261 -0.1499015 ]
 [-0.01286266 -0.12458052 -0.6341444 ]]
```

NumPy has built-in samplers of a lot of other common (and some not so common) distribution.

Array shape

Shape/reshape and multi-dimensional arrays.

Understanding the shape and reshaping of arrays is fundamental for many operations, especially when working with multi-dimensional arrays.

In our example, we're exploring the shape of arrays and how to reshape them:

- **1D array:** A simple 1-dimensional array with 4 elements is created with `np.array([1, 2, 3, 4])`. The shape of this array is (4,) because it has 4 elements in a single dimension.
- **1 × 4 array:** This array is created with `np.array([[1, 2, 3, 4]])`, explicitly defining it as a 2D array with 1 row and 4 columns. Its shape is (1, 4).
- **2 × 4 array:** A 2-dimensional array with 2 rows and 4 columns is created with `np.array([[1, 2, 3, 4], [5, 6, 7, 8]])`. Its shape is (2, 4).
- **Reshaping:** The command `array_1d.reshape(-1, 4)` reshapes the 1D array into a 2D array with 1 row and 4 columns. The -1 tells NumPy to infer the size of that dimension based on the total number of elements.
- **Size:** The `.size` attribute of an array returns the total number of elements in the array. In this case, for the `array_2by4`, it returns 8, indicating 8 elements in total.

```
[13]: array_1d=np.array([1,2,3,4])
      array_1by4=np.array([[1,2,3,4]])
      array_2by4=np.array([[1,2,3,4], [5,6,7,8]])

      print(array_1d.shape)
      print(array_1by4.shape)

      print(array_1d.reshape(-1, 4).shape)

      print(array_2by4.size) # projects: pytorch -- size similar to
                             numpy shape
```

```
(4,)
(1, 4)
(1, 4)
8
```

In this example, we create a large array with 400 elements (from 0 to 399) using a list comprehension. Then, we reshape it into a 20×20 matrix using the `.reshape()` method. After reshaping, the array becomes a 2D array with 20 rows and 20 columns. We then access all the elements in the 6th column (index 5, as indexing starts from 0) by using the slicing operator `[:,5]`.

```
[14]: larg_array=np.array([i for i in range(400)])
      larg_array=larg_array.reshape((20,20))

      print(larg_array[:,5])
```

```
[ 5  25  45  65  85 105 125 145 165 185 205 225 245 265 285
305 325 345
365 385]
```

Here's the examples demonstrate slicing and accessing elements in a 3D matrix:

```
[15]: larg_3d_array=np.array([i for i in range(1000)])
      larg_3d_array=larg_3d_array.reshape((10,10,10))
```

- The first example demonstrates how to access the elements in the 2nd row and 2nd column (index 1 for both) across all 10 layers of the 3D matrix.

```
[16]: print(larg_3d_array[:,1,1])
```

```
[ 11 111 211 311 411 511 611 711 811 911]
```

In this case, `:` selects all layers, and `1, 1` specifies the second row and second column across all layers.

- The second example shows how to access the 3rd row (index 2) across all columns and all layers.

```
[17]: print(larg_3d_array[2,:,1])
```

```
[201 211 221 231 241 251 261 271 281 291]
```

Here, `2` specifies the third row, `:` selects all columns, and `1` indicates we are selecting the second column.

- The third example shows how to access all elements in the 3rd row (index 2) and across all columns.

```
[18]: print(larg_3d_array[2,3,:])
```

```
[230 231 232 233 234 235 236 237 238 239]
```

Here, 2 specifies the third row, 3 specifies the fourth column, and : selects all layers.

- Finally, the fourth example demonstrates how to access the entire second row (index 1) across all columns and layers.

```
[19]: print(larg_3d_array[1,:,:])
```

```
[[100 101 102 103 104 105 106 107 108 109]
 [110 111 112 113 114 115 116 117 118 119]
 [120 121 122 123 124 125 126 127 128 129]
 [130 131 132 133 134 135 136 137 138 139]
 [140 141 142 143 144 145 146 147 148 149]
 [150 151 152 153 154 155 156 157 158 159]
 [160 161 162 163 164 165 166 167 168 169]
 [170 171 172 173 174 175 176 177 178 179]
 [180 181 182 183 184 185 186 187 188 189]
 [190 191 192 193 194 195 196 197 198 199]]
```

Here, 1 specifies the second row, and : selects all columns and layers.

Think about the order you need before using reshape.

Reshaping Arrays with Different Orders

Using `reshape` can be a bit tricky, so it's important to think about the order when reshaping an array.

There are two types of ordering in `reshape`:

- **Default (C order)** fills the array **row-wise**.
- **Specifying F order** fills the array **column-wise**.

In this example, we create a 1D array `saml1_array` with 4 elements.

```
[20]: saml1_array=np.arange(4)
```

Here we display the `saml1_array` after creating it.

```
[21]: samll_array
```

```
array([0, 1, 2, 3])
```

Here we reshape the array using the default row-major order (C).

```
[22]: print(np.reshape(samll_array, (2, 2), order='C')) # Default order
```

```
[[0 1]
 [2 3]]
```

Here we reshape the array using column-major order (F).

```
[23]: print(np.reshape(samll_array, (2, 2), order='F'))
```

```
[[0 2]
 [1 3]]
```

It's essential to choose the correct order depending on whether you want a row-wise or column-wise arrangement.

NumPy Math

NumPy provides a wide range of mathematical operations that work element-wise on arrays, eliminating the need for explicit loops. Since loops can be slow in Python, using NumPy's built-in functions improves efficiency.

This also works for sin, cos, tanh, etc.

```
[24]: array_1 = np.array([1,2,3,4])

# element wise operations!!

print(array_1 + 5) # Adds 5 to each element
print(array_1 * 5) # Multiplies each element by 5
print(np.sqrt(array_1)) # Square root of each element
print(np.power(array_1, 2)) # Each element raised to power 2
print(np.exp(array_1)) # Exponential function applied to each element
print(np.log(array_1)) # Natural logarithm of each element
```

```

[6 7 8 9]
[ 5 10 15 20]
[1.          1.41421356 1.73205081 2.          ]
[ 1  4  9 16]
[ 2.71828183  7.3890561  20.08553692 54.59815003]
[0.          0.69314718 1.09861229 1.38629436]

```

NumPy allows performing aggregation functions on arrays along specific axes (for sum, mean, avg, std, var, etc).

```

[25]: array_2d = np.array([[1,2,3], [4,5,6], [7,8,9]])

pp.pprint(array_2d)
print(f'shape={array_2d.shape}')

print(np.sum(array_2d)) # Sum of all elements
print(np.sum(array_2d, axis=0)) # Column-wise sum
print(np.sum(array_2d, axis=1)) # Row-wise sum

```

```

array([[1, 2, 3],
       [4, 5, 6],
       [7, 8, 9]])
shape=(3, 3)
45
[12 15 18]
[ 6 15 24]

```

NumPy also supports multi-dimensional arrays. The following example demonstrates summing elements along different axes in a **3D** array.

```

[26]: array_3d = np.array([i for i in range(8)]).reshape((2, 2, 2))
pp.pprint(array_3d)

print(np.sum(array_3d, axis=0)) # Sum along depth
print(np.sum(array_3d, axis=1)) # Sum along rows
print(np.sum(array_3d, axis=(1,2))) # Sum across multiple axes

```

```

array([[[0, 1],
       [2, 3]],

      [[4, 5],
       [6, 7]]])
[[ 4  6]
 [ 8 10]]

```



```
[[ 2  4]
 [10 12]]
[ 6 22]
```

Multiplication in NumPy

NumPy tends to perform operations element-wise. However, be careful when dealing with arrays of different dimensions, as this can lead to unexpected behavior. We will discuss this further in broadcasting, but always be mindful of array dimensions in general.

When two arrays have the same shape, element-wise multiplication is straightforward:

```
[27]: array_1 = np.array([1, 2, 3, 4])
      array_2 = np.array([3, 4, 5, 6])
```

```
[28]: print(array_1 * array_2) # element wise multiplication
```

```
[ 3  8 15 24]
```

This operation is equivalent to using `np.multiply`:

```
[29]: print(np.multiply(array_1, array_2))
```

```
[ 3  8 15 24]
```

Sometimes, dimensions may not match, causing issues in element-wise operations. Always check dimensions beforehand.

For example, reshaping `array_2` into a column vector:

```
[30]: print(array_2.reshape(4, -1))
```

```
[[3]
 [4]
 [5]
 [6]]
```

Checking the shape before and after reshaping:

```
[31]: print(array_2.reshape) # Output: (4,)
      print(array_2.reshape(4, -1).shape) # Output: (4,1)
```

```
(4,)
(4, 1)
```

If you multiply `array_1` by `array_2.reshape(4, -1)`, you get an outer product instead of element-wise multiplication:

```
[32]: print(array_1 * array_2.reshape(4, -1)) # come back to this later
```

```
[[ 3  6  9 12]
 [ 4  8 12 16]
 [ 5 10 15 20]
 [ 6 12 18 24]]
```

Dot product

The dot product can be done in multiple ways in NumPy, and the code demonstrates a few of these methods:

- **Element-wise Multiplication and Sum:** The first approach multiplies the corresponding elements of the two arrays (`array_1` and `array_2`) and then sums the results. This is the manual way of calculating the dot product:

```
[33]: # element wise multiplication --> sum
      print(np.sum(array_1 * array_2))
```

```
50
```

- **Using the `dot()` Method:** The second method uses NumPy's built-in `dot()` function to calculate the dot product directly:

```
[34]: print(array_1 .dot(array_2))
```

```
50
```

This method is more efficient and concise than element-wise multiplication and summation, but it performs the same operation internally.

- **Using `np.dot()` Function:** Another way to compute the dot product is by calling the `np.dot()` function explicitly:

```
[35]: print(np.dot(array_1, array_2))
```

50

- **Using the Matrix Multiplication Operator :** The `@` operator is another way to perform the dot product. It is equivalent to `np.matmul()`, which is used for matrix multiplication:

```
[36]: print(array_1 @ array_2) # equivalent to np.matmul(array_1, array_2)
```

50

This operator is commonly used when working with matrix multiplication, but for one-dimensional arrays, it gives the same result as the dot product.

```
[37]: print(array_1.shape)
```

(4,)

Here, you can't dot when the dimensions are incorrect. But it did not complain just now. Check the shapes!

```
[38]: array_1 = np.array([[1, 2, 3, 4]])
      array_2 = np.array([[3, 4, 5, 6]])

      print(array_1.shape)
      print(array_1.shape)

      print(array_1 * array_2)
      print(array_1.dot(array_2))
```

(1, 4)

(1, 4)

[[3 8 15 24]]

```
-----
ValueError                                Traceback (most recent call last)
Cell In[38], line 8
      5 print(array_1.shape)
      7 print(array_1 * array_2)
----> 8 print(array_1.dot(array_2))
```

```
ValueError: shapes (1,4) and (1,4) not aligned: 4 (dim 1) != 1 (dim 0)
```

With proper handling of shape, things work. Also, dot is just matrix multiplication. But behaves differently in higher dimensions! You might just want to write matrix multiply to keep things consistent and sure that you have the correct shape.

Difference Between Inner and Outer Product Using dot:

```
[39]: # T for transpose

print(array_1.dot(array_2.T)) # inner product
print(array_1.T.dot(array_2)) # outer product
```

```
[[50]]
[[ 3  4  5  6]
 [ 6  8 10 12]
 [ 9 12 15 18]
 [12 16 20 24]]
```

Using matmul Instead of dot:

```
[40]: print(np.matmul(array_1, array_2.T)) # inner product
print(np.matmul(array_1.T, array_2)) # outer product
```

```
[[50]]
[[ 3  4  5  6]
 [ 6  8 10 12]
 [ 9 12 15 18]
 [12 16 20 24]]
```

Note: matmul is preferred for higher-dimensional arrays.

Matrix Multiplication Example with Weights:

```
[41]: wight_matrix = np.array([1, 2, 3, 4]).reshape(2,2)
sample = np.array([[50,60]]).T # Transform to column vector

np.matmul(wight_matrix, sample)
```

```
array([[170],
       [390]])
```

Checking Dimensions Before Multiplication:

```
[42]: # sainty check

print(wight_matrix.shape)
print(sample.shape)
```

```
(2, 2)
(2, 1)
```

Output ensures multiplication is valid: $(2, 2) \times (2, 1)$ is valid.

And of course, we typically use `matmul` for **2D** matrix multiplications. For $\text{dim} > 3$, NumPy treats it as a stack of matrices.

```
[43]: mat1 = np.array([[1, 2], [3, 4]])
      mat2 = np.array([[5, 6], [7, 8]])

print(np.matmul(mat1, mat2))
```

```
[[19 22]
 [43 50]]
```

Notice that `np.multiply` is element-wise multiplication. NOT proper matrix multiplication.

```
[44]: a = np.array ([i for i in range(10)]).reshape(2,5)

print(a * a) # Element-wise multiplication
print(np.multiply(a, a)) # Also element-wise multiplication
print(np.multiply(a, 10)) # Multiply each element by 10
```

```
[[ 0  1  4  9 16]
 [25 36 49 64 81]]
[[ 0  1  4  9 16]
 [25 36 49 64 81]]
[[ 0 10 20 30 40]
 [50 60 70 80 90]]
```

II. Broadcasting and dimension manipulation

NumPy provides the capability to perform operations on arrays of different shapes by expanding or inferring dimensions as needed. This feature is called broadcasting.

Broadcasting is a powerful operation that allows NumPy to perform arithmetic operations on arrays of different shapes without explicitly reshaping them. For instance, if we have an array A of shape $(4, 1, 6, 1)$ and an array B of shape $(3, 1, 5)$, NumPy is able to automatically infer and expand the dimensions to make the operation possible.

Below are some examples to demonstrate this:

A	(4d array):	$8 \times 1 \times 6 \times 1$
B	(3d array):	$7 \times 1 \times 5$
Result	(4d array):	$8 \times 7 \times 6 \times 5$

A	(2d array):	5×4
B	(1d array):	1
Result	(2d array):	5×4

A	(2d array):	5×4
B	(1d array):	4
Result	(2d array):	5×4

A	(3d array):	$15 \times 3 \times 5$
B	(3d array):	$15 \times 1 \times 5$
Result	(3d array):	$15 \times 3 \times 5$

A	(3d array):	$15 \times 3 \times 5$
B	(3d array):	$15 \times 1 \times 5$
Result	(3d array):	$15 \times 3 \times 5$

A	(3d array):	$15 \times 3 \times 5$
B	(2d array):	3×5
Result	(3d array):	$15 \times 3 \times 5$

A	(3d array):	$15 \times 3 \times 5$
B	(2d array):	3×1
Result	(3d array):	$15 \times 3 \times 5$

Essentially, all dimensions of size 1 can be "overlooked" or "expanded" to match the dimension of another operator. However, the order of dimensions must be preserved. A dimension of size 1 can only be prepended, not appended. For example, the following would not work, even though one might think we can add another dimension at the end of B.

A (3d array): $15 \times 3 \times 5$
 B (2d array): 1×3
 Result (3d array): $15 \times 3 \times 5$

Heres a small Python code to demonstrate broadcasting:

- Creating 2D and 1D arrays and printing their values:

```
[45]: op1 = np.array([i for i in range (9)]).reshape(3, 3)
      op2 = np.array([[1, 2, 3]])
      op3 = np.array([1, 2, 3])

      pp.pprint(op1)
      pp.pprint(op2)
```

```
array([[0, 1, 2],
       [3, 4, 5],
       [6, 7, 8]])
array([[1, 2, 3]])
```

- Performing operations on arrays with different shapes:

```
[46]: # notice that the result here is DIFFERENT!

      print(op1.shape)
      print(op2.shape)
      print(op2.T.shape)
      pp.pprint(op1 + op2)
      pp.pprint(op1 + op2.T)
```

```
(3, 3)
(1, 3)
(3, 1)
array([[ 1,  3,  5],
       [ 4,  6,  8],
       [ 7,  9, 11]])
array([[ 1,  2,  3],
       [ 5,  6,  7],
       [ 9, 10, 11]])
```

- Performing operations on 1D arrays with 2D arrays:

[47]: *# notice that the result here is THE SAME!*

```
print(op3.shape)
print(op3.T.shape)
pp.pprint(op1 + op3)
pp.pprint(op1 + op3.T)
```

```
(3,)
(3,)
array([[ 1,  3,  5],
       [ 4,  6,  8],
       [ 7,  9, 11]])
array([[ 1,  3,  5],
       [ 4,  6,  8],
       [ 7,  9, 11]])
```

- Adding 3D array with 2D array (works when transposed):

Here, broadcasting won't work for $15 \times 3 \times 5$ with 1×3 . Because dimensions are only prepended.

But it will work for $15 \times 3 \times 5$ with 3×1 .

[48]:

```
op1 = np.array([i for i in range (225)]).reshape(15, 3, 5)
op2 = np.array([[1, 2, 3]])
```

```
# This does not work
# print (op1 + op2)
```

```
# This works
print(op1 + op2.T)
```

```
# BTW you can contract the cells by clicking on the left
```

```
[[[ 1  2  3  4  5]
   [ 7  8  9 10 11]
   [13 14 15 16 17]]
```

```
[[16 17 18 19 20]
 [22 23 24 25 26]
 [28 29 30 31 32]]
```

```
[[31 32 33 34 35]
 [37 38 39 40 41]]
```



```

[ 43  44  45  46  47]]

[[ 46  47  48  49  50]
 [ 52  53  54  55  56]
 [ 58  59  60  61  62]]

[[ 61  62  63  64  65]
 [ 67  68  69  70  71]
 [ 73  74  75  76  77]]

[[ 76  77  78  79  80]
 [ 82  83  84  85  86]
 [ 88  89  90  91  92]]

[[ 91  92  93  94  95]
 [ 97  98  99 100 101]
 [103 104 105 106 107]]

[[106 107 108 109 110]
 [112 113 114 115 116]
 [118 119 120 121 122]]

[[121 122 123 124 125]
 [127 128 129 130 131]
 [133 134 135 136 137]]

[[136 137 138 139 140]
 [142 143 144 145 146]
 [148 149 150 151 152]]

[[151 152 153 154 155]
 [157 158 159 160 161]
 [163 164 165 166 167]]

[[166 167 168 169 170]
 [172 173 174 175 176]
 [178 179 180 181 182]]

[[181 182 183 184 185]
 [187 188 189 190 191]
 [193 194 195 196 197]]

[[196 197 198 199 200]
 [202 203 204 205 206]

```

```
[208 209 210 211 212]]

[[211 212 213 214 215]
 [217 218 219 220 221]
 [223 224 225 226 227]]]
```

Tile

Treat broadcasting as tiling the lower dimensional array to suit the size of the "more complex" array.

In broadcasting, when we deal with arrays of different shapes, NumPy will often "expand" the smaller array to match the shape of the larger one. This process can be thought of as "tiling" the smaller array to fill the larger one.

Consider you have a simple 1D array and you want to "tile" it to create a larger array:

```
[49]: array = np.array([1, 2, 3])

# np.tile(array, shape)
print(np.tile(array, 2))
print(np.tile(array, (2, 3)))
```

```
[1 2 3 1 2 3]
[[1 2 3 1 2 3 1 2 3]
 [1 2 3 1 2 3 1 2 3]]
```

When working with multi-dimensional arrays, like `op1` and `op2`, the `np.tile` operation allows you to control how to replicate rows or columns. The following code demonstrates tiling of a smaller array across both axes:

Observe how, with transpose, the tile result is different, `Op2` originally has shape 1×3 , so:

Tiling (1×5) means tiling 2nd dimension 5 times yielding (1×15) .

Tiling the transpose, thus 3×1 , by (1×5) means tiling 2nd dimension 5 times, yielding (3×5) .

```
[50]: op1 = np.array([i for i in range(225)]).reshape(15, 3, 5)
      op2 = np.array([[1, 2, 3]])

      op_tiled = np.tile(op2, (1,5)) # Tiling in the second dimension
      print(op_tiled.shape) # (1, 15)

      op_tiled = np.tile(op2.T, (1, 5)) # Tiling the transpose in the
      second dimension
```

```
print(op_tiled.shape) # (3, 5)
```

```
(1, 15)
```

```
(3, 5)
```

Expand/Squeeze

The **expand** operation adds a new dimension with size 1 to an array. This is useful when you want to reshape an array to align with other operations without changing the actual values.

Here, we add a new dimension of size 1 to `op2` along the third axis using `np.expand_dims`. After adding the new dimension, we can use the `np.tile` function to replicate the `op_expanded` array into the shape (15, 3, 5) like this:

```
[51]: op_expanded = np.expand_dims(op2, axis=2)
      print(op_expanded.shape)

      op_tiled_2 = np.tile(op_expanded, (15, 1, 5))
      print(op_tiled_2.shape)
```

```
(1, 3, 1)
```

```
(15, 3, 5)
```

Same effect with `np.newaxis`. In the example below, we use `np.newaxis` to add new dimensions to the `op3` array:

```
[52]: op3 = np.array([i for i in range(9)]).reshape(3, 3)

      op_na = op3[np.newaxis, :]
      print(op_na)
      print(op_na.shape)

      op_na2 = op3[:, np.newaxis, :]
      print(op_na2)
      print(op_na2.shape)
```

```
[[[0 1 2]
    [3 4 5]
    [6 7 8]]]
```

```
(1, 3, 3)
```

```
[[[0 1 2]
```

```
    [3 4 5]]]
```

```
[[6 7 8]]  
(3, 1, 3)
```

The **squeeze** operation removes dimensions that have size 1. This operation is helpful when we want to reduce unnecessary dimensions.

In the example below, after expanding `op_expanded`, we use `np.squeeze` to remove the dimension with size 1:

```
[53]: print(op_expanded)  
      print(op_expanded.shape)  
  
      op_squeezed = np.squeeze(op_expanded)  
  
      print(op_squeezed)
```

```
[[[1]  
   [2]  
   [3]]]  
(1, 3, 1)  
[1 2 3]
```

Pairwise distance

Pairwise distance measures the distance between all pairs of points in a dataset. Here, we explore three methods to compute it efficiently using NumPy and SciPy.

```
[54]: samples = np.random.random((15, 5))  
      print(samples.shape)  
      print(samples)
```

```
(15, 5)  
[[0.75850727 0.36049141 0.93124085 0.63488671 0.58893675]  
 [0.32296072 0.88136613 0.16265247 0.71728334 0.42223944]  
 [0.72807863 0.70966103 0.93943362 0.46943724 0.56412865]  
 [0.17773388 0.5603522 0.78255727 0.46212297 0.23374694]  
 [0.62258692 0.60695462 0.6998574 0.86364396 0.75520547]  
 [0.06726005 0.91133302 0.82103424 0.82682636 0.12715513]  
 [0.36732864 0.0403257 0.46901371 0.08164388 0.58206681]  
 [0.91198243 0.94940969 0.26635842 0.91646861 0.65993981]  
 [0.96084127 0.28221982 0.56151111 0.4202182 0.6448163 ]  
 [0.7605314 0.7765274 0.31428235 0.07812688 0.12857528]
```

```
[0.11775609 0.50347117 0.20532918 0.73333503 0.24501654]
[0.40568698 0.76639631 0.29051708 0.55310733 0.34640104]
[0.55604453 0.29980905 0.87860219 0.68775208 0.31462201]
[0.20553709 0.98429845 0.22330205 0.51816665 0.36817271]
[0.54090057 0.46113351 0.13943321 0.94136331 0.5760766 ]]
```

Here are 3 ways to compute pairwise distances:

- **”Naive” method through tile expansion:** This method manually expands dimensions using `np.expand_dims`, then repeats them with `np.tile`. It calculates the difference and then applies `linalg.norm`. However, it is inefficient due to unnecessary memory usage.

```
[55]: # Without broadcasting
expanded1 = np.expand_dims(samples, axis=1)
tile1 = np.tile(expanded1, (1, samples.shape[0], 1))
#print(expanded1.shape)
#print(tile1.shape)
#print(tile1)

expanded2 = np.expand_dims(samples, axis=0)
tile2 = np.tile(expanded2, (samples.shape[0], 1, 1))
#print(expanded2.shape)
#print(tile2.shape)
#print(tile2)

diff = tile2 - tile1
distances = np.linalg.norm(diff, axis=-1)

#print(distances)
print(np.mean(distances))
```

```
0.7706383036526288
```

- **Convert the tile/expansion to broadcasting:** We leverage NumPy's broadcasting to compute pairwise differences efficiently, reducing memory usage.

```
[56]: # With broadcasting
diff = samples[:, np.newaxis, :] - samples[np.newaxis, :, :]
distances = np.linalg.norm(diff, axis=-1)

#print(distances)
```

```
print(np.mean(distances))
```

0.7706383036526288

- **SciPy one line:** SciPy provides an optimized way to compute pair-wise distances using `cdist` in a single line. The best and simplest option.

```
[57]: # With scipy
import scipy.spatial
distances = scipy.spatial.distance.cdist(samples, samples)
#print(distances)
#print(len(distances))
print(np.mean(distances))
```

0.7706383036526288

Vectorization

Vectorization is a technique that allows for optimizing code execution by replacing explicit loops with optimized matrix operations using NumPy. This leads to significant performance improvements, as operations are executed in parallel using low-level implementations.

`tqdm` is a nice package for you to track progress, or just kill time.

```
[58]: import time # time.time() gets wall time, time.clock() gets
processor time
from tqdm import tqdm
```

- **Dot Product:** NumPy is 25 times faster than loops here.

```
[59]: a = np.random.random(500000)
b = np.random.random(500000)

p_tic = time.perf_counter()
tic = time.time()

dot = 0.0;
for i in tqdm(range(len(a))):
    dot += a[i] * b[i]

print(dot)
```

```

toc = time.time()
p_toc = time.perf_counter()

print(f'Result: {dot}');
print(f'Compute time (wall): {round(1000 * (toc - tic),
6)}ms')
print(f'Compute time (cpu): {round(1000 * (toc - tic),
6)}ms\n')

#####

p_tic = time.perf_counter()
tic = time.time()

print(np.array(a).dot(np.array(b)))

toc = time.time()
p_toc = time.perf_counter()

print(f'(vectorized) Result: {dot}');
print(f'(vectorized) Compute time: {round(1000 * (toc -
tic), 6)}ms')
print(f'(vectorized) Compute time(cpu): {round(1000 *
(p_toc - p_tic), 6)}ms')

```

```
100%|| 500000/500000 [00:00<00:00, 588928.00it/s]
```

```
124949.5083837044
```

```
Result: 124949.5083837044
```

```
Compute time (wall): 851.000071ms
```

```
Compute time (cpu): 851.000071ms
```

```
124949.5083837044
```

```
(vectorized) Result: 124949.5083837044
```

```
(vectorized) Compute time: 3.000021ms
```

```
(vectorized) Compute time(cpu): 3.8ms
```

- **Matrix multiplication (2D):** NumPy is more than TWO THOUSAND times faster than loops here.

Matrix multiplication is a $O(n^3)$ complexity operation if implemented naively.

```
[60]: def matrix_mul(X, Y):
        # iterate through rows of X
        for i in range(len(X)):
            # iterate through columns of Y
            for j in range(len(Y[0])):
                # iterate through rows of Y
                for k in range(len(Y)):
                    result[i][j] += X[i][k] * Y[k][j]
        return result
```

```
[61]: X = np.random.random((200, 200))
        Y = np.random.random((200, 200))

        result = np.zeros((200, 200))

        # iterate through rows of X
        for i in tqdm(range(len(X))):
            # iterate through columns of Y
            for j in range(len(Y[0])):
                # iterate through rows of Y
                for k in range(len(Y)):
                    result[i][j] += X[i][k] * Y[k][j]

        s = np.sum(result)

        toc = time.time()
        p_toc = time.perf_counter()

        print(f'Result: {s}');
        print(f'Compute time (wall): {round(1000 * (toc - tic),
        6)}ms')
        print(f'Compute time (cpu): {round(1000 * (p_toc - p_tic),
        6)}ms\n')

        #####

        p_tic = time.perf_counter()
        tic = time.time()

        s = np.sum(result)
```



```

toc = time.time()
p_toc = time.perf_counter()

print(f'(vectorized) Result: {s}');
print(f'(vectorized) Compute time: {round(1000 * (toc -
tic), 6)}ms');
print(f'(vectorized) Compute time (cpu): {round(1000 *
(p_toc - p_tic), 6)}ms')

```

```
100%|| 200/200 [00:13<00:00, 15.33it/s]
```

```
Result: 1991930.484439076
```

```
Compute time (wall): 1058957.000017ms
```

```
Compute time (cpu): 1058956.8ms
```

```
(vectorized) Result: 1991930.484439076
```

```
(vectorized) Compute time: 0.0ms
```

```
(vectorized) Compute time (cpu): 0.4ms
```

- **Pairwise distance, again:** Again, NumPy is 30 times faster.

```

[62]: samples = np.random.random((100, 5))

p_tic = time.perf_counter()
tic = time.time()

total_dist = []
for s1 in samples:
    for s2 in samples:
        d = np.linalg.norm(s1 - s2)
        total_dist.append(d)

avg_dist = np.mean(total_dist)

toc = time.time()
p_toc = time.perf_counter()

print(f'Result: {avg_dist}');
print(f'Compute time (wall): {round(1000 * (toc - tic),
6)}ms');
print(f'Compute time (cpu): {round(1000 * (p_toc - p_tic),
6)}ms')

#####3

```

```

p_tic = time.perf_counter()
tic = time.time()

distances = np.linalg.norm(diff, axis=-1)
avg_dist = np.mean(distances)

toc = time.time()
p_toc = time.perf_counter()

print(f'Result: {avg_dist}');
print(f'Compute time (wall): {round(1000 * (toc - tic),
6)}ms')
print(f'Compute time (cpu): {round(1000 * (p_toc - p_tic),
6)}ms')

```

```

Result: 0.848083773681167
Compute time (wall): 69.999933ms
Compute time (cpu): 70.0ms

```

```

Result: 0.8668128310950173
Compute time (wall): 0.0ms
Compute time (cpu): 0.9ms

```

You might want to make sure that *OpenBLAS* is installed. *OpenBLAS* is a "basic linear algebra subprograms" package that basically, speeds up math for NumPy.

```
[63]: # np.show_config()
```

III. Matplotlib

Matplotlib is a plotting library. In this section give a brief introduction to the `matplotlib.pyplot` module, which provides a plotting system similar to that of MATLAB.

Matplotlib is a commonly used package for plotting in Python. While there are other libraries like Seaborn that provide more aesthetically pleasing visualizations, Matplotlib is sufficient for most tasks. You can fully customize your plots by specifying titles, axis ranges, legends, and colors. The `plt.plot()` function allows you to map x and y values easily, and you can save figures using `plt.savefig()`.

Simple plotting

We want to plot proper labels, series legend, and even markets.

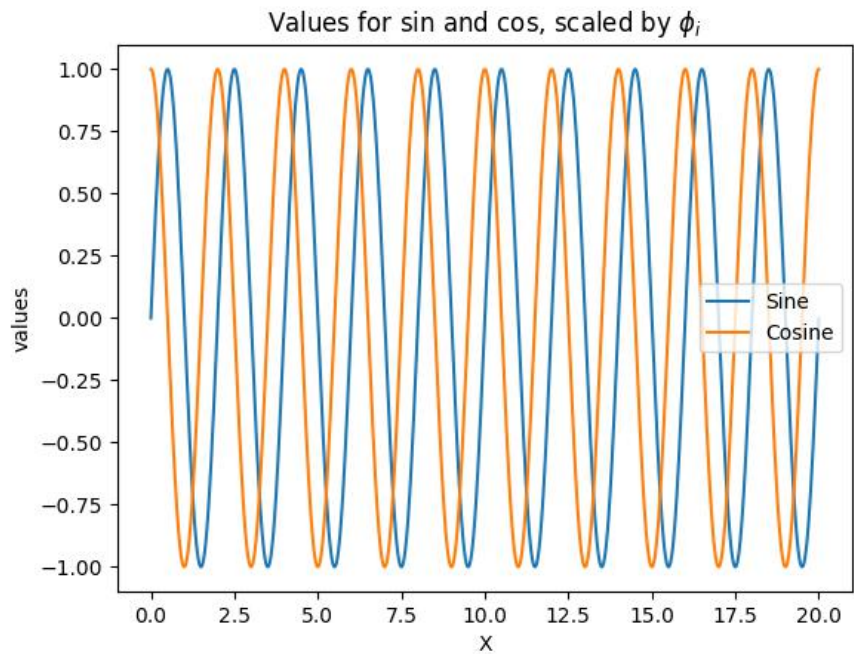
- **Importing:** The first step is to import Matplotlib.
- **Basic Usage:** The `plt.plot()` function is used to plot data on the x-axis and y-axis.
- **Customization:** You can specify the title, axis labels, legend, colors, and other elements to enhance the visualization.
- **Saving Figures:** To save a plot, use `plt.savefig('filename.jpg')`.

This example demonstrates plotting sine and cosine functions using Matplotlib, with various customizations applied to the graph.

```
[64]: # if you are using a headless environment. Very important if  
running on server  
# import matplotlib  
# matplotlib.use('Agg')  
  
import matplotlib.pyplot as plt
```

```
[65]: def draw_simple_sin_cos(x_values):  
  
    y1_values = np.sin(x_values * np.pi)  
    y2_values = np.cos(x_values * np.pi)  
  
    plt.plot(x_values, y1_values, label='Sine')  
    plt.plot(x_values, y2_values, label='Cosine')  
  
    plt.legend()  
    plt.xlabel('X')  
    plt.ylabel('values')  
    plt.title('Values for sin and cos, scaled by  $\phi_i$ ')
```

```
[66]: x_values = np.arange(0, 20, 0.001)  
  
draw_simple_sin_cos(x_values)  
plt.savefig('img1.jpg')  
plt.show()
```

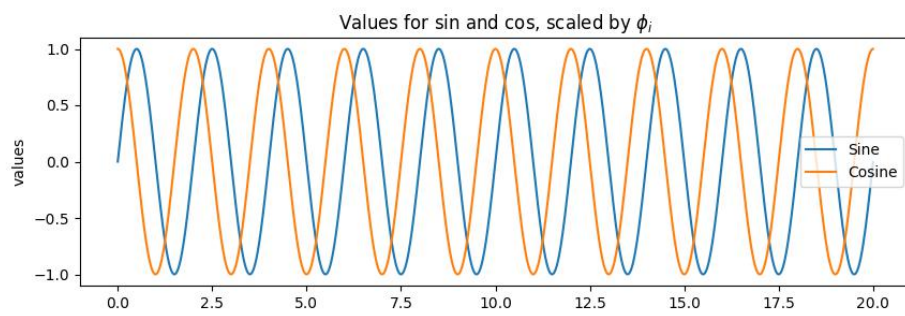


You can adjust figure size for aspect ratio then PDI for density of pixels. These combined give you resolution of the image.

```
[67]: plt.figure(figsize=(10,3), dpi=100) # 640 by 450

draw_simple_sin_cos(x_values)

plt.savefig('tutorial_sin.jpg')
plt.show()
```



Subplots in a grid can share axis labels through `sharex` and `sharey`.

```
[68]: def draw_subplot_sin_cos(index, x_values, ax):

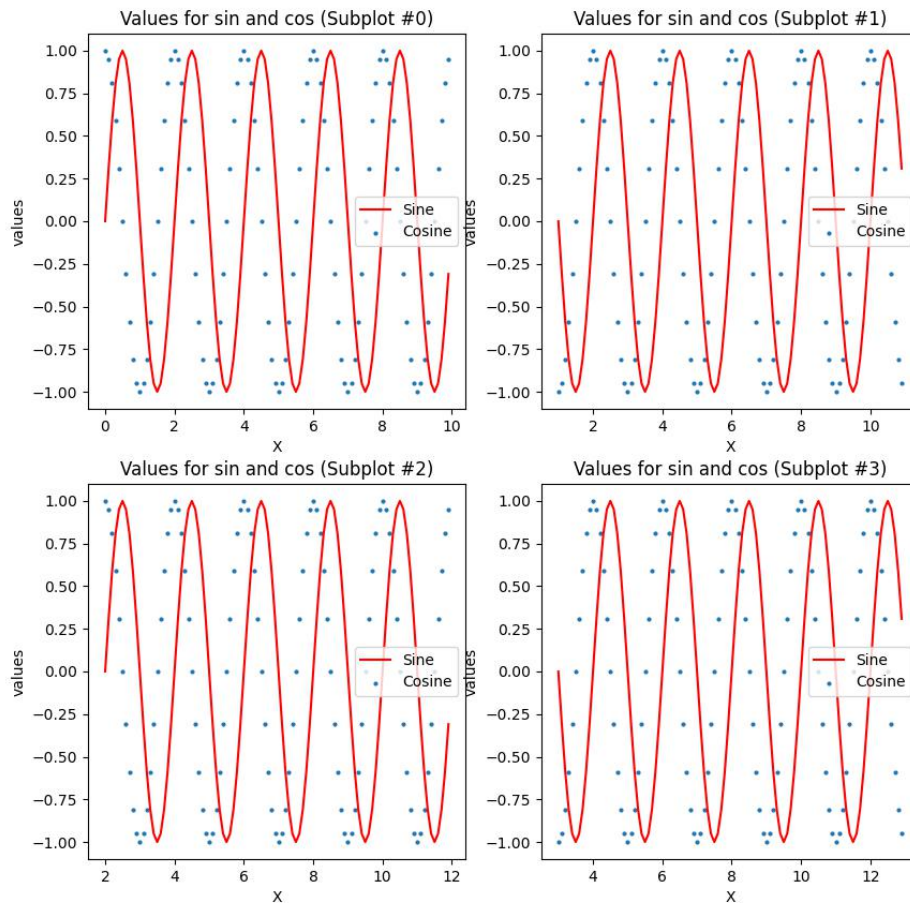
    y1_values = np.sin(x_values * np.pi)
    y2_values = np.cos(x_values * np.pi)

    ax.plot(x_values, y1_values, c='r', label='Sine')
    ax.scatter(x_values, y2_values, s=4, label='Cosine')

    ax.legend()
    ax.set_xlabel('X')
    ax.set_ylabel('values')
    ax.set_title(f'Values for sin and cos (Subplot #{index})')

[69]: fig, ax_list = plt.subplots(nrows=2, ncols=2, figsize=(10, 10))
# fig, ax_list = plt.subplots(nrows=2, ncols=2, sharex='col',
# sharey='row', figsize=(10, 10))

i = 0
for r, row in enumerate(ax_list):
    for c, ax in enumerate(row):
        x_values = np.arange(i, i + 10, 0.1)
        draw_subplot_sin_cos(i, x_values, ax)
        i += 1
plt.show()
```



Confusion matrix

Here we show plotting confusion matrix from scratch.

```
[70]: fig, ax = plt.subplots(figsize=(10, 10))

color = 'YlGn'
labels = ['Python', 'C++', 'Fortran']

cm = np.array([[0.7, 0.3, 0.2], [0.1, 0.5, 0.4], [0.05, 0.1,
0.85]])
heatmap = ax.pcolor(cm, cmap = color)
```

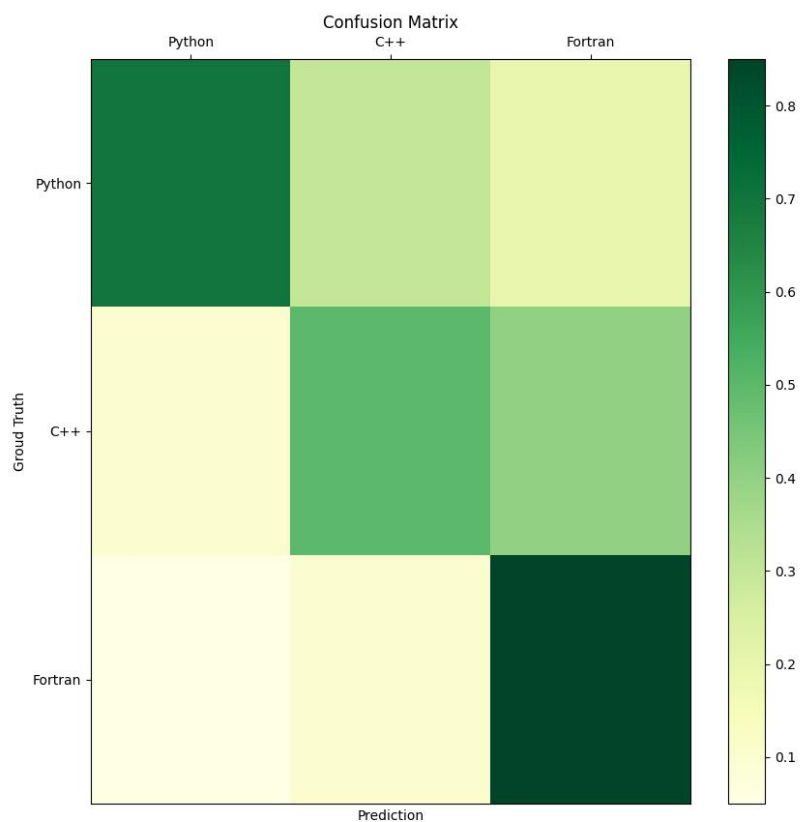
```

fig.colorbar(heatmap)
ax.invert_yaxis()
ax.xaxis.tick_top()

ax.set_title('Confusion Matrix')
ax.set_xlabel('Prediction')
ax.set_ylabel('Groud Truth')
ax.set_xticks(np.arange(cm.shape[0]) + 0.5, minor=False)
ax.set_yticks(np.arange(cm.shape[1]) + 0.5, minor=False)
ax.set_xticklabels(labels)
ax.set_yticklabels(labels)

plt.show()

```



Show image

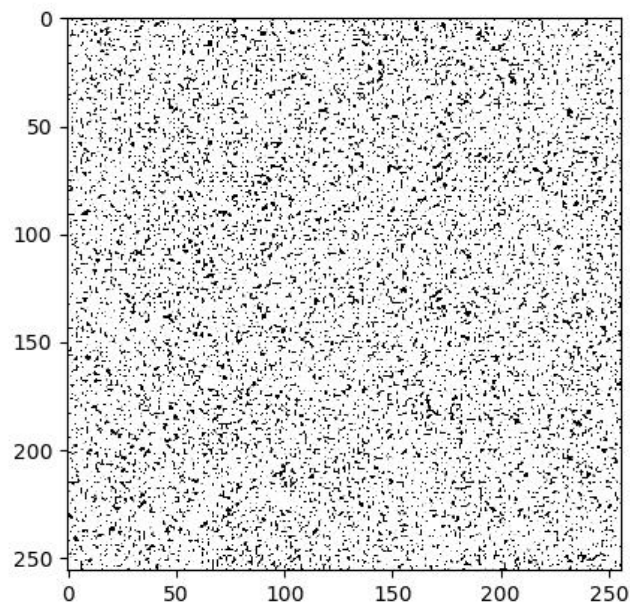
When showing image, remember to tell NumPy the range of pixel. Typically pixel values are either 0 – 1 or 0 – 255.

You can use `imshow()` to display images or even visualize random matrices as images.

```
[71]: img_arr = np.random.random((256, 256)) # 0>-1
      print(img_arr.shape)

      plt.imshow(img_arr, cmap='gray', vmin=0.2, vmax=0.25)
      plt.show()
```

(256, 256)

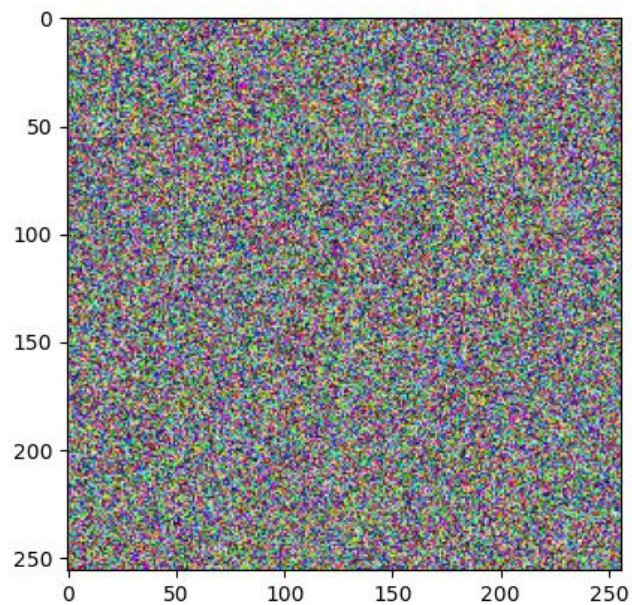


By default NumPy goes channel first.

```
[72]: img_arr = np.random.random((256, 256, 3)) # R, C, (RGB)
      print(img_arr.shape)

      plt.imshow(img_arr, vmin=0, vmax=1)
      plt.show()
```


(256, 256, 3)

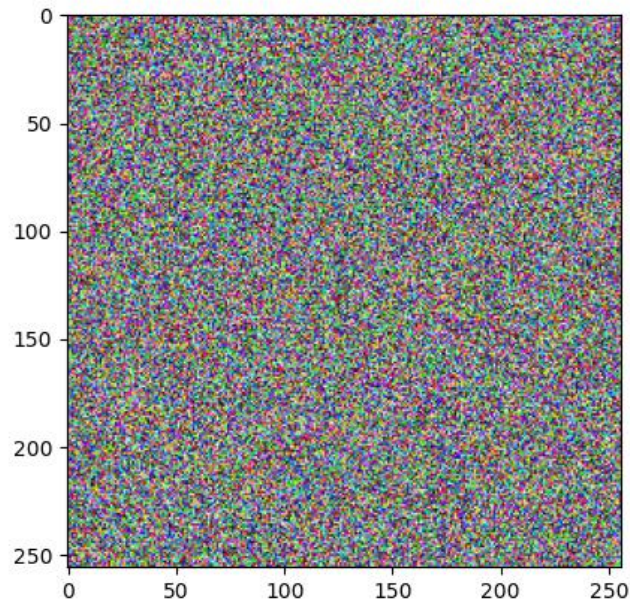


Remember to move axis around if you want to use the default plotting tool.

```
[73]: img_arr = np.random.random((3, 256, 256)) # (RGB) R C  
      print(img_arr.shape)  
  
      img_arr = np.moveaxis(img_arr, 0, -1)  
      print(img_arr.shape)  
  
      plt.imshow(img_arr, vmin=0, vmax=1)  
      plt.show()
```

(3, 256, 256)

(256, 256, 3)



Reading an image using `imageio` and displaying its dimensions. The `imageio` library allows reading images from files.

```
[74]: import imageio

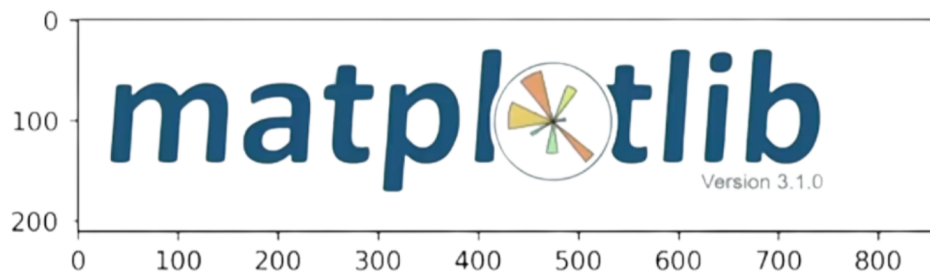
fname = 'sample.jpg' # File name of the image
img = imageio.imread(fname) # Read the image and load it as a
                             # NumPy array

print(img.shape) # Print the dimensions of the image
```

(211, 862, 3)

The image has a size of 211×862 pixels and consists of 3 color channels (RGB).

```
[75]: plt.figure(dpi = 250)
plt.imshow(img, vmin=0, vmax=1000, interpolation='bilinear')
plt.show()
```



Pandas

Pandas is great data processing library for table/database like data. Excellent for things came in or you wish to output as CSV/Excel.

```
[76]: import pandas as pd
```

```
[77]: data = pd.read_csv('train.csv')
data = data.dropna(axis=1, how='all') # Remove fully empty columns
data_short = data[:10]
```

```
[78]: data_short
```

	x_1	x_2	x_3	x_4	y
0	1	0	2.474657	0.345678	2
1	0	0	6.765348	0.239456	35
2	0	0	5.876438	0.875477	3
3	0	1	6.643469	2.474657	75
4	0	0	4.876536	6.765348	1
5	0	0	1.456780	5.876438	10
6	0	0	0.345678	6.643469	7
7	0	0	0.239456	0.239456	20
8	1	1	0.875477	0.875477	15
9	0	0	2.474657	6.643469	5

You can get basic statistic with little effort like this:

```
[79]: print(data['x_1'].describe()) #For one column

data.describe() # For the entire dataframe
```

```

count    51.000000
mean     0.215686
std      0.415390
min      0.000000
25%      0.000000
50%      0.000000
75%      0.000000
max      1.000000
Name: x_1, dtype: float64

```

	x_1	x_2	x_3	x_4	y
count	51.000000	51.000000	51.000000	51.000000	51.000000
mean	0.215686	0.254902	3.253765	2.645488	19.235294
std	0.415390	0.440143	3.091225	2.842181	20.872554
min	0.000000	0.000000	0.239456	0.239456	1.000000
25%	0.000000	0.000000	0.610577	0.345678	5.000000
50%	0.000000	0.000000	1.456788	0.875477	11.000000
75%	0.000000	0.500000	6.055503	5.376487	20.000000
max	1.000000	1.000000	9.656787	9.345678	75.000000

This line saves the data in `data_short` to a CSV file without including the index column.

```
[80]: data_short.to_csv('data_short.csv', index=False)
```

Manipulations

Columns can be selected and filtered based on value/name. Be careful with binary operation for filtering due to order of execution. Bit wise operation takes precedence over boolean.

```
[81]: data_short[['x_1', 'y']]
```

	x_1	y
0	1	2
1	0	35
2	0	3
3	0	75
4	0	1
5	0	10
6	0	7
7	0	20
8	1	15
9	0	5

To filter the rows in the `data_short` DataFrame where the value of `y` is greater than 5 and the value of `x_3` is less than 1.5, you can use the following code:

```
[82]: data_short[(data_short['y']>5) & (data_short['x_3']<1.5)]
```

	x_1	x_2	x_3	x_4	y
5	0	0	1.456780	5.876438	10
6	0	0	0.345678	6.643469	7
7	0	0	0.239456	0.239456	20
8	1	1	0.875477	0.875477	15

This code creates a subset of the DataFrame that only includes rows that meet both conditions. The `&` operator is used to combine the two conditions, ensuring both must be true for a row to be included.

A filter can be applied to generate a new column (you can use this to apply a trained model for prediction result).

We can add column based on file/conditions.

```
[83]: def filter_func(row):
```

```

    if row['x_1'] == 1.0 and row['x_2'] == 0.0:
        return row['y'] * 10

    return -1

data_short['new_column'] = data_short[['x_1', 'x_2',
'y']].apply(filter_func, axis=1)

data_short

```

	x_1	x_2	x_3	x_4	y	new_column
0	1	0	2.474657	0.345678	2	20
1	0	0	6.765348	0.239456	35	-1
2	0	0	5.876438	0.875477	3	-1
3	0	1	6.643469	2.474657	75	-1
4	0	0	4.876536	6.765348	1	-1
5	0	0	1.456780	5.876438	10	-1
6	0	0	0.345678	6.643469	7	-1
7	0	0	0.239456	0.239456	20	-1
8	1	1	0.875477	0.875477	15	-1
9	0	0	2.474657	6.643469	5	-1

Iterating through Pandas rows can be done as follows. Each row is "dictionary". Adding data directly via a list of values is also valid.

```

[84]: col2 = []
      for i, row in data_short.iterrows():
          print(f'Row {i}: y-value: {row["y"]}')
          col2.append(row['y'] ** 2)

      data_short['col_2'] = col2
      data_short

```

```

Row 0: y-value: 2.0
Row 1: y-value: 35.0
Row 2: y-value: 3.0
Row 3: y-value: 75.0
Row 4: y-value: 1.0
Row 5: y-value: 10.0
Row 6: y-value: 7.0
Row 7: y-value: 20.0
Row 8: y-value: 15.0

```

Row 9: y-value: 5.0

	x_1	x_2	x_3	x_4	y	new_column	col.2
0	1	0	2.474657	0.345678	2	20	4.0
1	0	0	6.765348	0.239456	35	-1	1225.0
2	0	0	5.876438	0.875477	3	-1	9.0
3	0	1	6.643469	2.474657	75	-1	5625.0
4	0	0	4.876536	6.765348	1	-1	1.0
5	0	0	1.456780	5.876438	10	-1	100.0
6	0	0	0.345678	6.643469	7	-1	49.0
7	0	0	0.239456	0.239456	20	-1	400.0
8	1	1	0.875477	0.875477	15	-1	225.0
9	0	0	2.474657	6.643469	5	-1	25.0

Not a great example here, but `loc` means index by value, `iloc` means index by index. For example, you can do `iloc` by -1, but Not `loc` by -1.

```
[85]: print(data.loc[9])
      print(data.iloc[-1])
```

```
x_1    0.000000
x_2    0.000000
x_3    2.474657
x_4    6.643469
y       5.000000
Name: 9, dtype: float64
x_1     1.000000
x_2     0.000000
x_3     9.345678
x_4     4.876536
y      11.000000
Name: 50, dtype: float64
```

Create dataframe

You can create dataframe from dictionary in row or column major manner. Notice that "extra" things will be filled with `NaN`.

```
[86]: data_list = [{'a': i, 'b': i + 1} for i in range(15)]
      data_list[5] = {'a': 10, 'b': 9, 'c': -1}

      df = pd.DataFrame(data_list)
```

df

	a	b	c
0	0	1	NaN
1	1	2	NaN
2	2	3	NaN
3	3	4	NaN
4	4	5	NaN
5	10	9	-1.0
6	6	7	NaN
7	7	8	NaN
8	8	9	NaN
9	9	10	NaN
10	10	11	NaN
11	11	12	NaN
12	12	13	NaN
13	13	14	NaN
14	14	715	NaN

Dataframe can also be created from 2D array. Naming the rows and columns is a good practice.

```
[87]: data_2d = np.array([i for i in range(50)]).reshape(5, 10)

df = pd.DataFrame(data_2d, columns = [f'col {i}' for i in
range(10)], index=[f'row{i}' for i in range (5)])
df
```

	col 0	col 1	col 2	col 3	col 4	col 5	col 6	col 7	col 8	col 9
row0	0	1	2	3	4	5	6	7	8	9
row1	10	11	12	13	14	15	16	17	18	19
row2	20	21	22	23	24	25	26	27	28	29
row3	30	31	32	33	34	35	36	37	38	39
row4	40	41	42	43	44	45	46	47	48	49

Similarly, you can create dataframe directly from dictionary. It also supports whether the dictionary keys are row/col indices.

```
[88]: data_dict = {'col 1': [3, 2, 1, 0],
                  'col 2': ['a', 'b', 'c', 'd']}
```



```
df = pd.DataFrame.from_dict(data_dict)
df
```

	col 1	col 2
0	3	a
1	2	b
2	1	c
3	0	d

```
[89]: df = pd.DataFrame.from_dict(data_dict, orient='index')
df
```

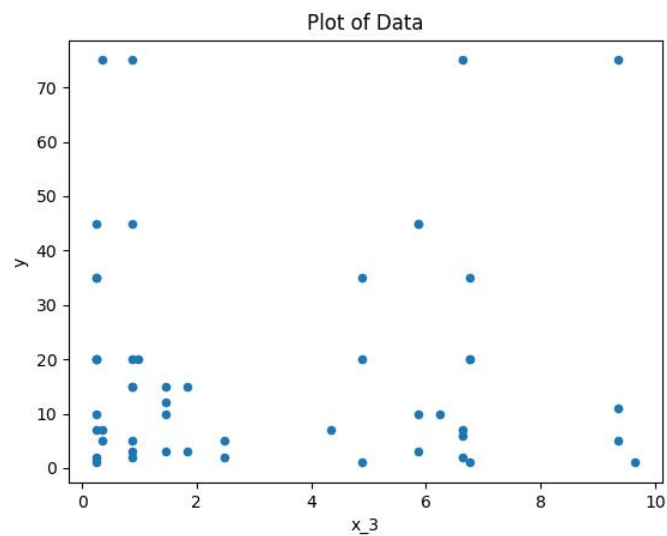
	0	1	2	3
col 1	3	2	1	0
col 2	a	b	c	d

Simple plotting

Pandas also support plotting. The images it generates are the same style as those in Matplotlib. Pandas plotting provides a quick way to visualize, while you might still need to resort to Matplotlib for more formal plots with higher flexibility.

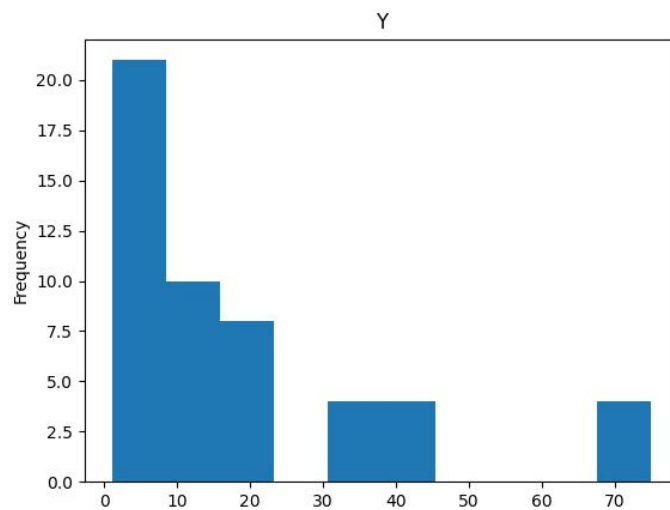
In the example below, a **scatter plot** is created using Pandas' `.plot()` method. The `kind='scatter'` argument specifies the plot type, while `x='x_3'` and `y='y'` define the x and y axes, respectively. The `title` parameter sets a title for the plot.

```
[90]: data.plot(kind='scatter', x='x_3', y='y', title='Plot of Data');
```



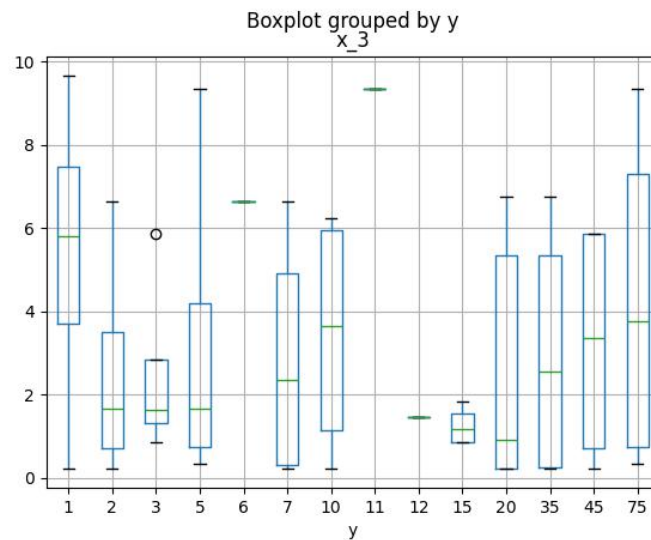
In the example below, a histogram is created for the y column. The height of each bar represents the frequency of values within a specific range. This allows for a better understanding of how the values in y are distributed.

```
[91]: data['y'].plot(kind='hist', title='Y')
```



In the example below, a box plot is generated for the column `x_3`, grouped by the values in the `y` column. This helps in understanding how the distribution of `x_3` varies across different categories of `y`.

```
[92]: data.boxplot (column='x_3', by='y');
```



Pandas provides the `to_numpy()` method to convert a `DataFrame` into a NumPy array. This is useful when performing numerical computations, as NumPy arrays offer better performance and efficiency compared to Pandas `DataFrames` for certain operations.

In the example below, the entire `DataFrame` `data` is converted into a NumPy array, allowing for further processing using NumPy functions.

```
[93]: data.to_numpy()
```

```
array([[ 1.          ,  0.          ,  2.474657 ,  0.345678 ,  2.
],
       [ 0.          ,  0.          ,  6.765348 ,  0.239456 , 35.
],
       [ 0.          ,  0.          ,  5.876438 ,  0.875477 ,  3.
],
       [ 0.          ,  1.          ,  6.643469 ,  2.474657 , 75.
],
       [ 0.          ,  0.          ,  4.876536 ,  6.765348 ,  1.
],
       ...])
```

```

[ 0.      , 0.      , 1.45678 , 5.876438 , 10.
],
[ 0.      , 0.      , 0.345678 , 6.643469 , 7.
],
[ 0.      , 0.      , 0.239456 , 0.239456 , 20.
],
[ 1.      , 1.      , 0.875477 , 0.875477 , 15.
],
[ 0.      , 0.      , 2.474657 , 6.643469 , 5.
],
[ 0.      , 0.      , 6.765348 , 4.876536 , 20.
],
[ 0.      , 1.      , 5.876438 , 1.465678 , 45.
],
[ 0.      , 1.      , 6.643469 , 0.345678 , 2.
],
[ 1.      , 1.      , 4.876536 , 0.239456 , 35.
],
[ 0.      , 0.      , 1.845678 , 0.875477 , 3.
],
[ 0.      , 0.      , 0.345678 , 0.875477 , 75.
],
[ 1.      , 0.      , 0.239456 , 0.875477 , 1.
],
[ 0.      , 0.      , 0.239456 , 0.875477 , 10.
],
[ 0.      , 1.      , 0.239456 , 6.765348 , 7.
],
[ 0.      , 0.      , 0.239456 , 5.876438 , 20.
],
[ 0.      , 0.      , 0.875477 , 6.643469 , 15.
],
[ 0.      , 1.      , 0.875477 , 4.876536 , 5.
],
[ 1.      , 1.      , 0.875477 , 1.845678 , 20.
],
[ 0.      , 1.      , 0.875477 , 0.345678 , 45.
],
[ 0.      , 1.      , 0.875477 , 0.239456 , 2.
],
[ 1.      , 0.      , 0.245679 , 0.239456 , 35.
],
[ 0.      , 0.      , 1.456788 , 0.239456 , 3.
],

```

```

[ 0.      , 0.      , 9.345678 , 0.239456 , 75.
],
[ 1.      , 0.      , 6.765348 , 0.875477 , 1.
],
[ 0.      , 0.      , 5.876438 , 0.875477 , 10.
],
[ 0.      , 1.      , 6.643469 , 0.239456 , 7.
],
[ 0.      , 0.      , 4.876536 , 0.239456 , 20.
],
[ 0.      , 0.      , 1.845678 , 0.239456 , 15.
],
[ 0.      , 0.      , 0.345678 , 0.239456 , 5.
],
[ 0.      , 0.      , 0.239456 , 0.875477 , 20.
],
[ 0.      , 1.      , 0.239456 , 0.875477 , 45.
],
[ 1.      , 0.      , 0.239456 , 0.875477 , 2.
],
[ 0.      , 0.      , 0.239456 , 0.875477 , 35.
],
[ 0.      , 0.      , 0.875477 , 0.875477 , 3.
],
[ 0.      , 0.      , 0.875477 , 0.245679 , 75.
],
[ 0.      , 0.      , 9.656787 , 1.456788 , 1.
],
[ 1.      , 0.      , 6.234567 , 9.345678 , 10.
],
[ 0.      , 0.      , 4.345678 , 6.765348 , 7.
],
[ 0.      , 0.      , 0.987886 , 5.876438 , 20.
],
[ 1.      , 1.      , 1.456788 , 6.643469 , 15.
],
[ 0.      , 0.      , 9.345678 , 4.876536 , 5.
],
[ 0.      , 0.      , 6.765348 , 2.2345678 , 20.
],
[ 0.      , 0.      , 5.876438 , 0.864367 , 45.
],
[ 0.      , 0.      , 6.643469 , 9.285678 , 6.
],

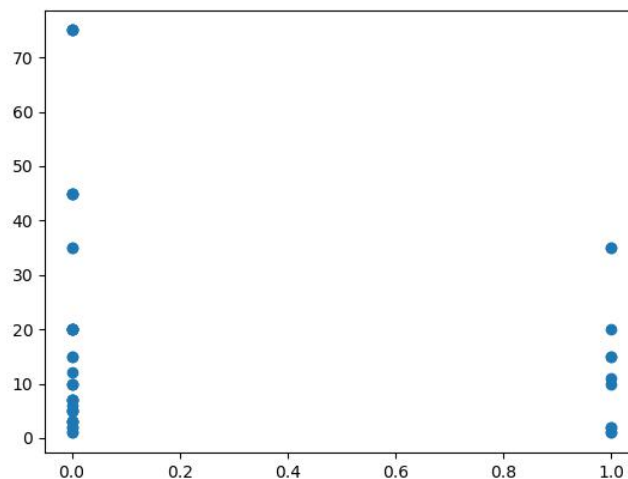
```

```
[ 0.          ,  0.          ,  1.456788 ,  6.643469 , 12.
 ],
 [ 1.          ,  0.          ,  9.345678 ,  4.876536 , 11.
 ]])
```

Matplotlib provides the `scatter()` function to create scatter plots, which are useful for visualizing relationships between two numerical variables.

In the example below, a scatter plot is generated using the `x_1` column for the x-axis and the `y` column for the y-axis. Each point in the plot represents a data entry, helping to identify patterns, trends, or potential correlations between the two variables.

```
[94]: plt.scatter(data['x_1'], data['y'])
```



IV. Summary

NumPy is an essential tool for anyone working with numerical data in Python. Its capabilities for array manipulation, efficient mathematical operations, and performance optimization provide significant advantages over traditional programming methods. Key practices include:

- Always import NumPy and familiarize yourself with array creation and data types.
- Leverage NumPy's vectorized operations to enhance performance and reduce computation time.

- Utilize reshaping and broadcasting features to handle arrays of different dimensions efficiently.
- Regularly check dimensions and data types during debugging to avoid errors in calculations.
- Consider integrating NumPy with optimized libraries for improved performance in large-scale applications.
- **Mastering Visualization and Data Handling:** Familiarity with matplotlib and pandas is crucial for effective data analysis and presentation in Python. These libraries provide a robust framework for visualizing data and performing complex manipulations.

Convenient math functions, read before use!

Python Command	Description
<code>np.linalg.inv</code>	Inverse of a matrix (NumPy equivalent)
<code>np.linalg.eig</code>	Get eigenvalues (Read documentation on <code>eig</code> and NumPy equivalent)
<code>np.matmul</code>	Matrix multiplication
<code>np.zeros</code>	Create a matrix filled with zeros (Read on <code>np.ones</code>)
<code>np.arange</code>	Start, stop, step size (Read on <code>np.linspace</code>)
<code>np.identity</code>	Create an identity matrix
<code>np.vstack</code>	Vertically stack two arrays (Read on <code>np.hstack</code>)
<code>array.shape</code>	Get shape of numpy array
<code>array.dtype</code>	Check data type of array (for precision, for weird behavior)
<code>type(stuff)</code>	Get type of a variable
<code>import pdb; pdb.set_trace()</code>	Set a breakpoint (https://docs.python.org/3/library/pdb.html)
<code>print(f'Myname is {name}')</code>	Easy way to construct a message

Table 1: NumPy Commands and Their Descriptions